PID CONTROL FOR 3-DOF ROBOT

A MINI PROJECT REPORT

Submitted by

KIRAN JOY T

TCR24ECRA13

to

the A P J Abdul Kalam Technological University

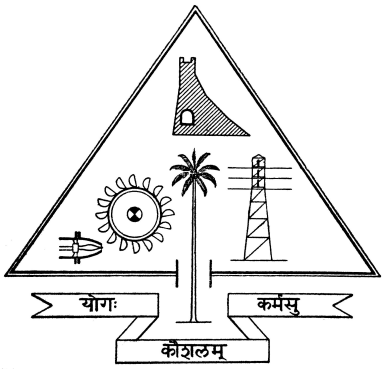
in partial fulfillment of the requirements for the award of the Degree

of

MASTER OF TECHNOLOGY

in

ROBOTICS AND AUTOMATION



DEPT. OF ELECTRONICS & COMMUNICATION ENGINEERING

GOVERNMENT ENGINEERING COLLEGE

THRISSUR-680009

July 2025

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Prof. Shamla B**, Assistant Professor, Department of Electronics and Communication Engineering, Government Engineering College Thrissur, for her valuable guidance, encouragement, and unwavering support throughout the course of this mini-project work.

I am also thankful to **Dr. Roy Francis**, Head of the Department, and our project coordinators **Dr. A.R. Jayan** and **Prof. Riyas K.S.** for their continuous support and valuable suggestions.

My heartfelt thanks to all the faculty members and staff of the Department of Electronics and Communication Engineering, GEC Thrissur, for their assistance and cooperation during the project.

Finally, I would like to extend my deepest appreciation to my family and friends for their constant motivation and encouragement throughout the course of this work.

Kiran Joy T

DECLARATION

I hereby declare that the mini project report **PID CONTROL FOR 3-DOF ROBOT**, submitted for partial fulfillment of the requirements for the award of the degree of Master of Technology of the APJ Abdul Kalam Technological University, Kerala, is a bonafide work done by me under supervision of **Prof. Shamla B**.

This submission represents my ideas in my own words, and where ideas or words of others have been included, I have adequately and accurately cited and referenced the sources. I also declare that I have adhered to the ethics of academic honesty and integrity and have not misrepresented or fabricated any data idea, fact, or source in my submission. I understand that any violation of the above will be a cause for disciplinary action by the institute and/or the University and can also evoke penal action from the sources that have thus not been properly cited or from whom proper permission has not been obtained. This report has not previously formed the basis for the award of any degree, diploma, or similar title at any other University.

Kiran Joy T
Department of Electronics and
Communication Engineering
Government Engineering College
Thrissur-680009, Kerala

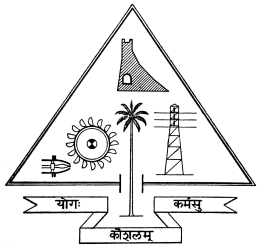
Thrissur
July 2025

DEPT. OF ELECTRONICS & COMMUNICATION ENGINEERING

GOVERNMENT ENGINEERING COLLEGE

THRISSUR

2024 - 2025



CERTIFICATE

This is to certify that the project final report entitled **"PID CONTROL FOR 3-DOF ROBOT"** submitted by **KIRAN JOY T (TCR24ECRA13)**, to the APJ Abdul Kalam Technological University in partial fulfillment of the M.Tech. degree in Robotics and Automation is a bonafide record of the project work carried out by him under my guidance and supervision. This report in any form has not been submitted to any other University or Institute for any purpose.

Prof. Shamla B
(Project Guide)
Assistant Professor
Dept. of ECE
Government Engineering College, Thrissur

Dr. Roy Francis
(Head of the Department)
Associate Professor
Dept. of ECE
Government Engineering College, Thrissur

Dr. A.R. Jayan
(Project Co-ordinator)
Professor
Dept. of ECE
Government Engineering College, Thrissur

Prof. Riyas K S
(Project Co-ordinator)
Associate Professor
Dept. of ECE
Government Engineering College, Thrissur

ABSTRACT

This mini-project explores the design, simulation, and hardware implementation of a Proportional–Integral–Derivative (PID) control system for a 3-Degree-of-Freedom (3-DOF) robotic arm. The modeling and control logic were developed in MATLAB and Simulink, enabling accurate joint trajectory simulation under closed-loop feedback. SolidWorks was used to design the robotic structure, while Arduino served as the control interface for real-time actuation. Due to limitations in sensor feedback integration, the physical hardware operates in an open-loop mode, relying on pre-defined input values rather than real-time correction. Despite this, the project establishes a functional and scalable framework for robotic joint control and provides a foundation for future improvements, including sensor-based closed-loop operation using potentiometers or inertial measurement units (IMUs), with the goal of enhancing motion precision and system autonomy.

CONTENTS

ACKNOWLEDGMENT **i**

ABSTRACT **ii**

LIST OF FIGURES **iv**

Chapter 1. INTRODUCTION **1**

1.1 Background of the Project 1

1.2 Objective of the Project 2

Chapter 2. LITERATURE REVIEW **3**

2.1 Development of 3-DOF Robotic Arms 3

2.2 Classical Control Approaches in Robotics 3

2.3 Simulation Environments for Control Design 4

2.4 Arduino and Open-Loop Control Limitations 4

Chapter 3. PROBLEM STATEMENT **5**

Chapter 4. TOOLS AND PACKAGES USED **6**

4.1 Software Tools 6

4.2 Hardware Platform 7

4.3 MATLAB Support Packages 7

4.4 Additional Tools and Components 7

Chapter 5. DESIGN AND METHODOLOGY **9**

5.1 Mechanical Design of the Robotic Arm 9

5.2 Control System Design Using Simulink 10

5.3 Hardware Implementation in Open-Loop Mode 11

5.4 Power Supply and Buck Converter Integration 12
5.5 Workflow Integration Between Simulation and Hardware . . 12

Chapter 6. IMPLEMENTATION **14**
6.1 Hardware Setup 14
6.2 Software and Communication 14
6.3 Signal and Control Integration 15
6.4 Mechanical Design Transfer 15
6.5 Testing and Validation 15

Chapter 7. RESULTS & DISCUSSION **17**
7.1 Simulation Outcomes 17
7.2 Physical Implementation Observations 17
7.3 Comparison of Simulation and Hardware Performance . . . 18
7.4 Practical Limitations 19

Chapter 8. CONCLUSION & FUTURE SCOPE **20**
8.1 Conclusion 20
8.2 Future Scope 20

REFERENCES **22**

LIST OF FIGURES

5.1 CAD model exported to MATLAB using the Simscape Multibody Link plugin 10
5.2 PID control architecture- Closed-loop feedback system 11
5.3 Arduino IDE 12

7.1 Simulation of 3-DOF Robotic Arm 18
7.2 3-DOF Robotic Arm controlled by Arduino Microcontroller 19

Chapter 1

INTRODUCTION

Automation and robotics are rapidly transforming traditional systems by enhancing precision, flexibility, and reliability in both industrial and research applications. Among various robotic configurations, robotic arms with multiple joints are available due to their versatility in performing mechanical tasks. This project involves a 3-degree-of-freedom (3-DOF) robotic arm, focusing on its motion control using classical control theory and software tools.

The project leverages MATLAB and Simulink to simulate joint behavior and implement a control strategy. Arduino is used to handle physical actuation. SolidWorks supports the mechanical design of the arm. Although simulation employs a feedback-based control loop to monitor and correct joint positions, the actual hardware runs in open-loop mode, limited by the absence of real-time sensors. Despite this, the system provides a working foundation for exploring motion control, with future reference for developing a closed-loop control using feedback sensors such as IMUs or potentiometers.

1.1 BACKGROUND OF THE PROJECT

Robotic arms play a key role in performing repetitive and precise tasks in environments where manual operation is inefficient or unsafe. A system with 3 degrees of freedom offers basic rotational and positional control, making it a practical platform for understanding fundamental motion principles and control algorithms.

In this project, a PID (Proportional–Integral–Derivative) controller is se-

lected for managing the movement of each joint. This controller helps reduce errors between desired and actual positions by adjusting control inputs accordingly. The simulation of the robotic arm behavior is determined using MATLAB's Simulink environment, which provides dynamic testing of the control logic under virtual conditions.

While the simulated environment benefits from real-time feedback for performance tuning, the hardware prototype is operated using predefined control inputs due to the absence of sensing components. Still, this implementation helps demonstrate the core principles of motion control and sets the stage for incorporating sensor-based feedback in future iterations.

1.2 OBJECTIVE OF THE PROJECT

The main goals of the project are to:

- Design the 3-DOF robotic arm structure using CAD tools.
- Develop and simulate a PID control method for precise joint control in Simulink.
- Develop the control model into hardware using Arduino-based actuation.
- Operate the hardware under open-loop conditions to observe system response.
- Analyze the differences in behavior between simulation (closed-loop) and hardware (open-loop).

Chapter 2

LITERATURE REVIEW

Controlling the motion of robotic arms is a well-studied area in the field of robotics and automation. Various researchers and engineers have contributed to improving robotic arm design and performance by integrating control systems, simulation tools, and microcontroller-based actuation. This section provides a review of the foundational topics relevant to this project, including robotic arm mechanisms, control strategies such as PID, simulation environments, and hardware-based execution, with a focus on systems operated without real-time feedback.

2.1 DEVELOPMENT OF 3-DOF ROBOTIC ARMS

The 3-DOF robotic arms are available for applications that require basic movement and orientation. These systems are providing sufficient flexibility for tasks such as sorting, positioning, or pick-and-place operations. The simplicity in structure makes them ideal for testing control logic and mechanical behavior in laboratory environments. The design includes a rotating base and two joints that allow vertical and angular movements. SolidWorks or Fusion 360 is the CAD tool used for designing mechanical structures in several academic prototypes.

2.2 CLASSICAL CONTROL APPROACHES IN ROBOTICS

Among the available control strategies, PID control remains one of the most widely applied methods for robotic motion. It is known for its balance of simplicity and effectiveness in minimizing the error between a desired and actual posi-

tion. Although more sophisticated control algorithms exist, such as model predictive control or neural-based methods, PID control remains dominant for small- to medium-scale robotic systems due to its low computational cost and ease of implementation. The control parameters, such as P, I, and D, can be optimized to improve responsiveness, reduce overshoot, and improve steady-state accuracy.

2.3 SIMULATION ENVIRONMENTS FOR CONTROL DESIGN

Simulation environments play a vital role in verifying control strategies before applying them to real-world systems. Tools such as MATLAB and Simulink provide modelling and visualization in a virtual environment. It also applies the control algorithm. These platforms offer flexibility in modifying parameters and observing system behavior under different conditions. The ability to connect simulation models with embedded systems also enables a smooth transition from software testing to physical hardware integration.

2.4 ARDUINO AND OPEN-LOOP CONTROL LIMITATIONS

Microcontrollers such as Arduino are popular in educational and prototyping projects due to their affordability and ease of use. They allow users to implement control logic in physical systems using motors and basic electrical components. However, when such systems operated in open-loop mode—without feedback from sensors—the control becomes less reliable. External disturbances, load variations, or mechanical inaccuracies may lead to incorrect positioning. Despite this, open-loop systems are valuable for initial testing and educational purposes. They provide a foundation upon which closed-loop systems with real-time feedback can operate in the future.

Chapter 3

PROBLEM STATEMENT

The use of robotic manipulators is becoming increasingly common in applications where repetitive or precise motion is essential. However, accurately managing the movement of multiple joints in such systems remains a key technical challenge, particularly when control feedback is limited. In academic and prototype-level projects, achieving this control often depends on straightforward algorithms such as PID control, which, while reliable, can be limited in effectiveness without proper sensor feedback.

In many basic robotic systems, including those controlled using microcontrollers like Arduino, the absence of real-time position data restricts the system to an open-loop mode. This setup lacks corrective feedback, which results in reduced precision. And provides inconsistent performance while trying to follow a specific movement path. Bridging the gap between simulation and physical hardware shows difficulties. While working in simulation, it provides a feedback loop that the hardware does not provide.

This project aims to address the practical problem of implementing a joint control strategy for a 3-degree-of-freedom robotic arm, in which the control logic is developed and tested in MATLAB and Simulink. The physical system operated through Arduino in an open-loop configuration. The primary focus is on building a system that, despite the absence of feedback, can execute basic motions reliably while also providing a clear path for future enhancement with closed-loop control using sensors.

Chapter 4

TOOLS AND PACKAGES USED

This section outlines the software tools, hardware platforms, and support packages used for simulating, designing, and implementing the control system for a 3-DOF robotic arm.

4.1 SOFTWARE TOOLS

- **MATLAB**:- MATLAB was introduced as the primary platform to develop the control algorithm and simulation for the system. It provides a versatile environment for mathematical modeling, algorithm development, and data visualization. The PID controller can be designed and tuned with the help of MATLAB's Control Toolbox.

- **Simulink**:- Simulink, integrated within MATLAB, was used for graphical modeling and simulation of the robotic arm. It allowed for the creation of block diagrams representing the plant, controller, and input/output systems. Simulink provides real-time simulation capabilities, which help to evaluate the system response under different PID parameters.

- **SolidWorks**:- SolidWorks provides the design for the mechanical structure of the robotic arm. The 3D CAD model helped visualize the dimensions, link lengths, and movements of each joint before physical assembly.

4.2 HARDWARE PLATFORM

- **Arduino UNO**:- The Arduino UNO board served as the interface between the MATLAB environment and the physical hardware. It was assigned to drive servo motors based on control signals sent from MATLAB or predefined within the Arduino sketch.

- **Servo Motors (MG996R)**:- Three identical servo motors are used at each joint and at the base of the robotic arm. These motors receive PWM signals from Arduino to rotate to specified angles.

- **Buck Converter**:- A buck converter, also known as a step-down DC-DC converter, is an efficient power regulation circuit used to reduce a higher DC voltage to a lower DC voltage level. In the context of this project, the buck converter steps down the 12V power supply to a suitable voltage (typically 5V–6V) required for powering microcontrollers and servo motors.

4.3 MATLAB SUPPORT PACKAGES

- **Simscape Multibody Link**:- A plug-in for SolidWorks that enables direct export of CAD assemblies into Simscape Multibody format (.XML and .SLDPRT/.SLDASM files). These exported files can be converted into Simscape blocks using smimport in MATLAB

4.4 ADDITIONAL TOOLS AND COMPONENTS

- **Jumper wires and breadboard**:– For creating electrical connections between the Arduino and other components.

- **Power supply (5V/12V)**:– Used to power the servo motors externally to prevent overloading the Arduino.

- **USB Cable**:– For programming and serial communication between the computer and Arduino board.

Chapter 5

DESIGN AND METHODOLOGY

The development of a PID-controlled 3-DOF robotic arm involved a systematic approach that covered modeling, simulation, hardware integration, and power regulation. The methodology was divided into two core phases: a simulation phase based on MATLAB/Simulink and a hardware implementation phase using Arduino and servo motors. While the simulation employed closed-loop PID feedback for joint control, the hardware prototype operated in an open-loop configuration due to the absence of real-time sensing components. This section presents the detailed design workflow, control strategy, and the integration of mechanical and electronic components.

5.1 MECHANICAL DESIGN OF THE ROBOTIC ARM

The mechanical structure of the robotic arm was designed using SolidWorks. The design consisted of three degrees of freedom corresponding to a base rotational joint, a shoulder joint, and an elbow joint. Each joint provided a single rotational axis, giving the arm the ability to position its end-effector within a two-dimensional workspace while maintaining rotational capability at the base. SolidWorks was not only used for visualization but also to ensure appropriate dimensions, joint limits, and physical constraints. The complete CAD model was then exported to MATLAB using the Simscape Multibody Link plugin. This allowed seamless integration of the mechanical structure into the Simulink environment for dynamic simulation and control development.

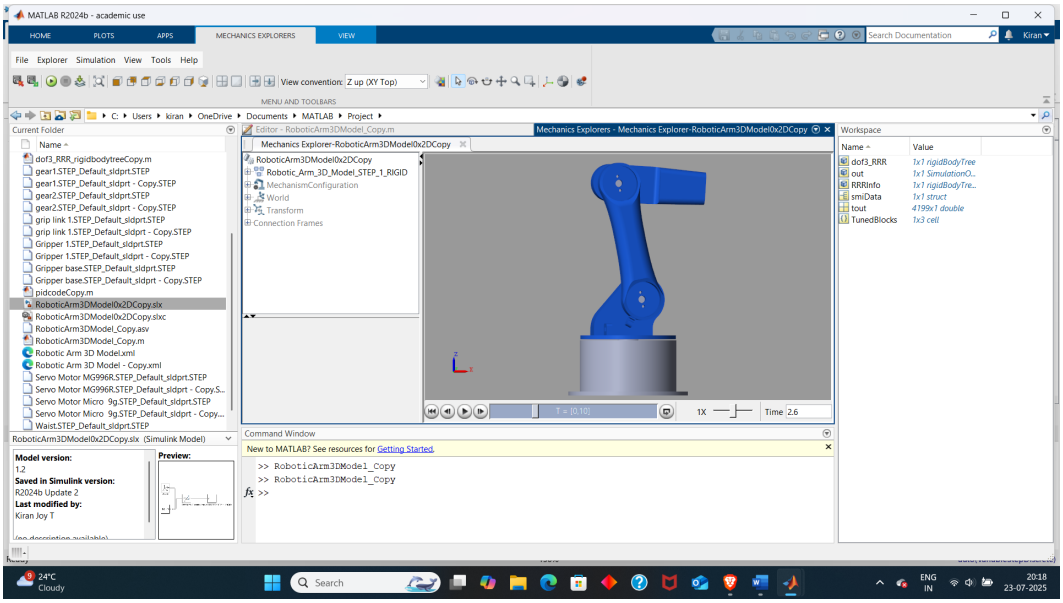


Figure 5.1: CAD model exported to MATLAB using the Simscape Multibody Link plugin

5.2 CONTROL SYSTEM DESIGN USING SIMULINK

In the simulation phase, MATLAB Simulink was used to develop a PID control architecture for each joint of the robotic arm. The control system was implemented as a closed-loop feedback system where the desired angular position of each joint was compared with the simulated joint output, and the resulting error was fed to a PID controller. Tuning of the proportional, integral, and derivative gains was performed iteratively to achieve an optimal response in terms of stability, rise time, and steady-state error. The simulation environment was configured using Simscape Multibody blocks to realistically model joint dynamics and link interactions. The output of the controller was converted into angular values, which were later mapped to PWM signals for the physical servos.

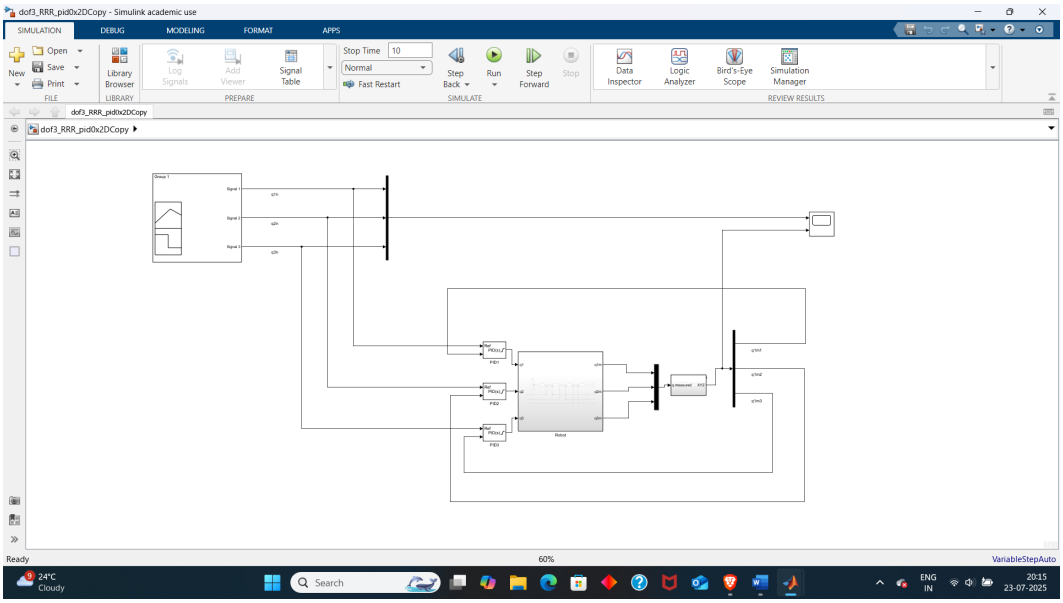


Figure 5.2: PID control architecture- Closed-loop feedback system

5.3 HARDWARE IMPLEMENTATION IN OPEN-LOOP MODE

The hardware realization of the robotic arm was executed using an Arduino Uno board and three MG996R servo motors. Due to the unavailability of real-time joint angle sensors such as potentiometers or encoders, the hardware system functioned as an open-loop controller. The target joint angles obtained from the Simulink simulation were converted to appropriate PWM signals and programmed into the Arduino sketch. Upon uploading the code, the servos moved to the specified angles without receiving any feedback about the actual arm position. Although this limited the system's ability to self-correct, it provided a basic functional demonstration of joint movement corresponding to simulation results. The use of the open-loop approach simplified the hardware but at the cost of reduced accuracy and robustness.

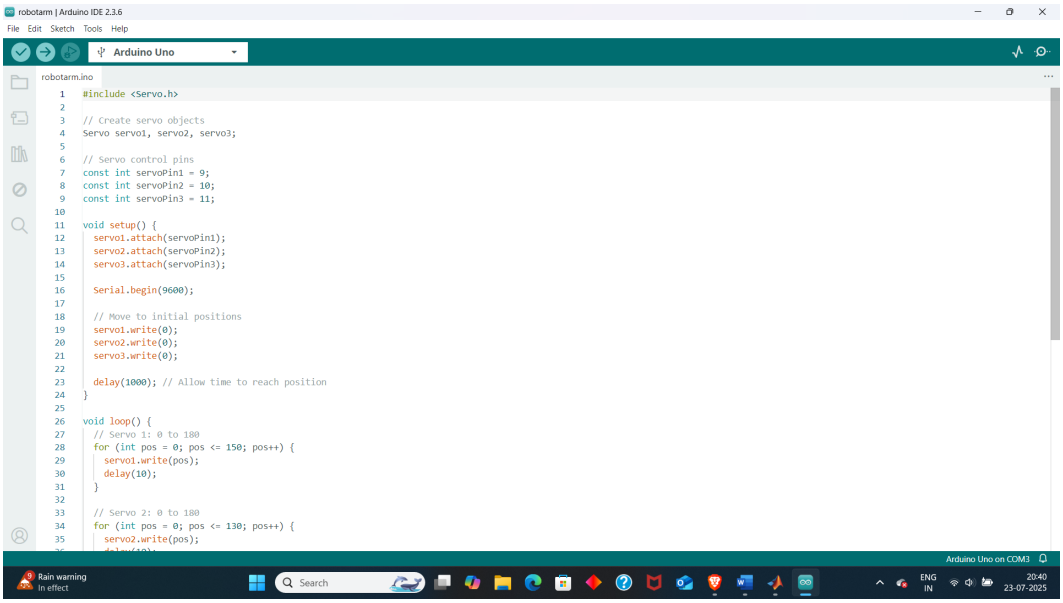


Figure 5.3: Arduino IDE

5.4 POWER SUPPLY AND BUCK CONVERTER INTEGRATION

The entire system was powered by a 12V sealed lead-acid (SLA) battery. Since the servo motors and the Arduino board require a regulated voltage of around 5–6V, a buck converter module (LM2596) was used. The buck converter efficiently stepped down the voltage from 12V to a suitable level for the servos and the Arduino. Proper wiring and voltage checks were performed to ensure stable power delivery, and precautions were taken to avoid overcurrent or voltage fluctuations that could damage the electronics. The use of a dedicated buck converter enhanced the safety and performance of the actuation system by isolating the power control from the logic circuit.

5.5 WORKFLOW INTEGRATION BETWEEN SIMULATION AND HARDWARE

The overall workflow of the project involved translating simulated control data into real-world joint actuation. In the simulation stage, joint angles were gen-

erated in response to PID control commands. These angular values were then extracted and used to determine equivalent PWM signals compatible with Arduino-controlled servo motors. The Arduino code was manually adjusted with these values to approximate the desired behavior in the physical setup. While real-time communication between MATLAB and Arduino was not implemented, this manual data transfer allowed for partial validation of the simulation results in the physical domain.

Chapter 6

IMPLEMENTATION

6.1 HARDWARE SETUP

The physical realization of the 3-DOF robotic arm was carried out using three MG996R servo motors. Each motor was assigned to one joint: the base, shoulder, and elbow, allowing independent movement for each axis. The mechanical structure was designed and assembled using lightweight materials, and the arm configuration followed a revolute joint mechanism. A 12V DC power supply was used in combination with a buck converter to ensure stable 5–6V output for the servo motors. The entire system was powered through a regulated source to avoid voltage fluctuations that could affect motor performance. An Arduino Uno board was used as the main controller, responsible for receiving position data and generating PWM signals to drive the servo motors.

6.2 SOFTWARE AND COMMUNICATION

The control logic for the robotic arm was developed in MATLAB/Simulink. Using the Simulink Support Package for Arduino Hardware, a communication link was established between MATLAB and the Arduino board. This setup allowed position commands generated in MATLAB to be transmitted to the servos via Arduino. Although the simulation was developed using a closed-loop PID controller, the hardware implementation followed an open-loop configuration due to the absence of real-time feedback elements like encoders or IMUs. The desired angular

positions were sent directly to the servos without any correction mechanism based on sensor feedback.

6.3 SIGNAL AND CONTROL INTEGRATION

The PID controller in Simulink was tuned to achieve smooth joint movement in simulation. For hardware execution, the final command angles from the controller were passed as input signals to the Arduino, which then generated the necessary PWM outputs. The servo angles were controlled using the Servo.h library in Arduino, which interprets angle data and maps it into timed pulses. The system lacks position sensors, so each servo responds to the command blindly, without verifying actual joint positions. This highlights the limitation of open-loop control in practical applications.

6.4 MECHANICAL DESIGN TRANSFER

The mechanical design of the 3-DOF robotic arm was created in SolidWorks. The design was then exported as a CAD assembly and imported into MATLAB using Simscape Multibody. This allowed for the validation of kinematic behavior and dynamic interaction in a virtual environment. Although the hardware model operated without real-time simulation, the exported model was used to visualize joint movements and verify system dynamics prior to deployment.

6.5 TESTING AND VALIDATION

After uploading the control logic to Arduino, the robot was tested for joint range and motion response. Each servo motor was evaluated individually, and then the entire sequence was tested to perform combined joint movements. The open-loop nature of the hardware limited its ability to respond to disturbances, but it

successfully executed predefined angle commands. The system showed consistent performance under normal load, confirming that the model and controller design were reliable for open-loop demonstrations.

Chapter 7

RESULTS & DISCUSSION

7.1 SIMULATION OUTCOMES

The MATLAB and Simulink-based simulation of the 3-DOF robotic arm yielded consistent and controlled joint movements in response to reference inputs. The arm's base, shoulder, and elbow joints were each managed by individual PID controllers, whose parameters were tuned to balance quick responsiveness with minimal overshoot. The simulation provided insight into system dynamics, and the visual feedback from the Simscape Multibody model verified that the arm's links moved as expected in a realistic environment. No oscillations or control instability were observed during simulation, indicating that the designed PID control strategy was effective for position tracking tasks within the simulated setup.

7.2 PHYSICAL IMPLEMENTATION OBSERVATIONS

When shifting to hardware, the robotic arm operates through an Arduino-based platform in an open-loop setup, which means that no position feedback is used. Predefined angular commands were issued to the servo motors, and the motors attempted to reach these angles without any correction based on actual joint position. While this method lacks adaptability to real-world disturbances or load changes, the arm performed acceptably in basic tasks with stable loads. Each motor moved to the expected position based on command inputs, although slight deviations were observed due to torque limitations or mechanical play. These issues are

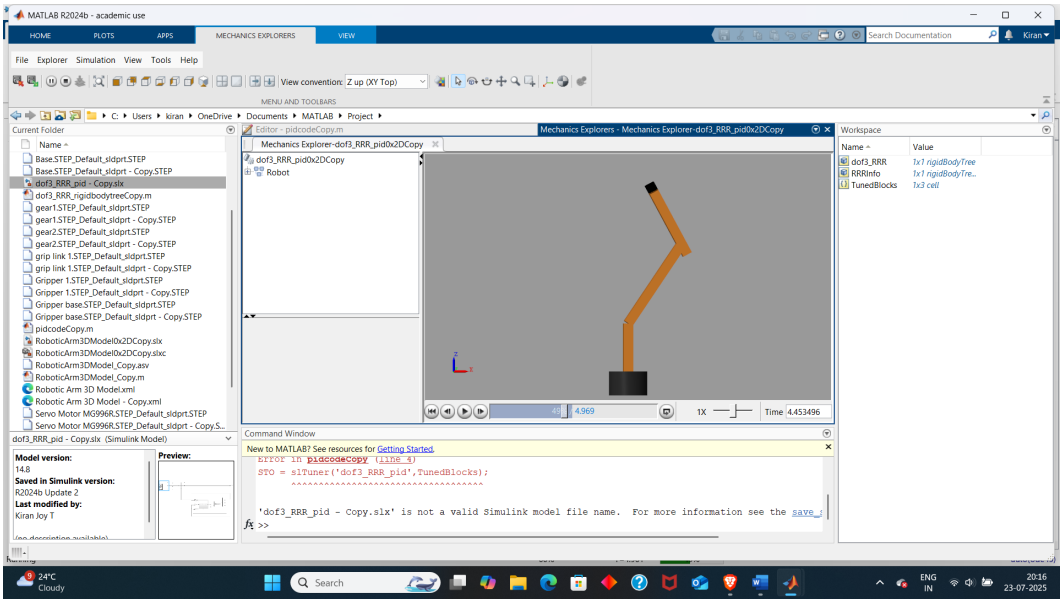


Figure 7.1: Simulation of 3-DOF Robotic Arm

common in open-loop systems and highlight the potential benefits of sensor integration in future iterations.

7.3 COMPARISON OF SIMULATION AND HARDWARE PERFORMANCE

The simulation and physical trials provided valuable insights. In the virtual environment, closed-loop feedback ensured continuous correction and precise joint control. On the other hand, the open-loop nature of the hardware meant that accuracy relied solely on motor calibration and power stability. This comparison revealed that while open-loop control can serve as a proof-of-concept, real-time feedback systems offer clear advantages in handling unexpected variations, achieving better positional accuracy, and maintaining consistent performance under load.

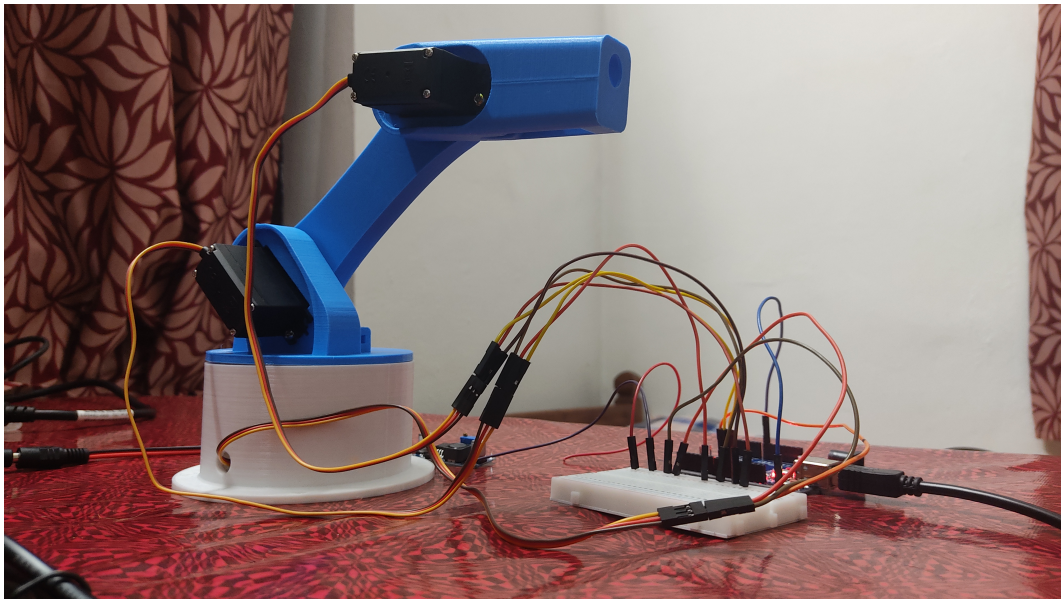


Figure 7.2: 3-DOF Robotic Arm controlled by Arduino Microcontroller

7.4 PRACTICAL LIMITATIONS

During hardware testing, several practical challenges became evident. The servos show inconsistent behavior under higher loads or extended operation. The buck converter, while effective in regulating voltage, sometimes led to voltage drops during peak current demands, which affected the stability of the servo output. Additionally, the absence of sensors meant that errors in joint angles could not be corrected automatically. These findings indicate that while the system meets control objectives, a closed-loop design would significantly enhance performance for real-world applications.

Chapter 8

CONCLUSION & FUTURE SCOPE

8.1 CONCLUSION

This project explored the development and implementation of a PID control strategy for a 3-DOF robotic arm using both simulation and hardware platforms. MATLAB and Simulink were used for modeling and tuning the PID controllers, allowing for closed-loop performance testing in a controlled environment. The hardware implementation employed an Arduino-based setup in an open-loop configuration due to the absence of joint feedback sensors. While the system performed basic motion tasks effectively, it also highlighted the limitations of open-loop control in terms of accuracy and adaptability.

Despite these constraints, the project achieved its primary objectives of demonstrating the feasibility of PID control for joint angle manipulation and validating system behavior across both virtual and real-world setups. The simulation results showed reliable and smooth joint responses, while the hardware model operated within acceptable bounds of movement, albeit without real-time correction.

8.2 FUTURE SCOPE

Building on this foundation, several enhancements can be pursued in future work:

- **Closed-Loop Hardware Control**:- Integrating sensors such as potentiometers or IMUs will enable real-time feedback, allowing precise joint angle

monitoring and error correction.

- **Advanced Control Strategies**:- Techniques such as adaptive control, fuzzy logic, or neural network-based controllers can be explored to improve robustness under variable loads and conditions.

- **Wireless Communication**:- Implementing wireless modules (e.g., Bluetooth, Wi-Fi) could allow for remote operation and monitoring.

- **Mechanical Improvements**:- Enhancing structural rigidity, joint precision, and torque capabilities would allow the arm to handle more complex tasks and payloads.

- **Integration with Computer Vision**-: Coupling the robotic arm with a vision system could enable autonomous object detection, sorting, or pick-and-place applications.

- **Human–Machine Interface (HMI)**:- Development of a GUI or voice control interface would improve user interaction and accessibility for non-technical users.

These potential upgrades would not only increase the versatility and reliability of the robotic system but also align it with real-world industrial and assistive robotic applications.

REFERENCES

[1] **Kuo, B. C., & Golnaraghi, F.**, (2014). Automatic Control Systems (9th ed.). Wiley. — A foundational text for control theory, including PID control.

[2] **Ogata, K.**, (2010). Modern Control Engineering (5th ed.). Prentice Hall. — Comprehensive coverage of classical and modern control techniques.

[3] **Spong, M. W., Hutchinson, S., & Vidyasagar, M.**, (2005). Robot Modeling and Control. Wiley. — Offers detailed modeling techniques and control strategies for robotic arms.

[4] **Jain, R. K.**, (2018). Modern Digital Electronics (5th ed.). McGraw-Hill Education. — Useful for understanding interfacing components like PWM and servo motors.

[5] **Niku, S. B.**, (2010). Introduction to Robotics: Analysis, Control, Applications (2nd ed.). Wiley. — Practical insight into robotic design, simulation, and control algorithms.

[6] **Kumar, S., & Kaur, R.** (2020). "Design and Simulation of PID Controlled 3-DOF Robotic Arm using MATLAB/Simulink." International Journal of Engineering Research & Technology (IJERT), 9(6), 123–128. — Research reference for simulation of 3-DOF arms with PID logic.

[7] **Ghosh, P. K.**,(2021). "Arduino Based Robotic Arm Control Using PID Technique." International Journal of Innovative Technology and Exploring Engineering (IJITEE), 10(4), 67–70.

[8] **Arduino.**,(2023). Arduino Uno Rev3 — Product Reference. Retrieved from https://www.arduino.cc - Hardware reference used for implementation.

[9] **MathWorks.**,(2023). PID Controller Design with MATLAB and Simulink. Retrieved from https://www.mathworks.com — Official documentation on PID implementation and tuning using MATLAB/Simulink.